Inteligência Artificial

Inteligência Artificial

Samy Soares
samy@ufc.br

Outras Buscas

Objetivos

- Introduzir métodos heurísticos diversos;
- Comparar diferentes técnicas de busca quanto à sua eficiência e confiabilidade.

Roteiro

- Buscas Não-Informadas:
  - Profundidade Limitada / Iterativa
  - Bidirecional
- Buscas Informadas:
  - Pelo Melhor Primeiro
  - Com Limite Superior

Buscas Não-Informadas

Profundidade Limitada

- Variação da Busca em Profundidade (BP)

Profundidade Limitada

- Variação da Busca em Profundidade (BP)
  - Utilizada para árvores muito profundas ou ilimitadas
    - Caminhos são expandidos somente até profundidade ***l***.
    - Visa rapidez de resposta.

Profundidade Limitada

- O limite de profundidade:
  - Impede caminhos ilimitados (cíclicos);

Profundidade Limitada

- O limite de profundidade:
  - Impede caminhos ilimitados (cíclicos);
  - Pode levar à incompletude:
    - Suponha profundidade ***l*** = 10 e objetivo no nível ***d*** > 11.
    - Ocorre quando não conhecemos a profundidade onde se encontram os estados objetivo.

Profundidade Limitada

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
   return RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
   cutoff_occurred? ← false
   if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
   else if DEPTH[node] = limit then return cutoff
   else for each successor in EXPAND(node, problem) do
      result ← RECURSIVE-DLS(successor, problem, limit)
      if result = cutoff then cutoff_occurred? ← true
      else if result ≠ failure then return result
   if cutoff_occurred? then return cutoff else return failure
```

Profundidade Limitada

- Exemplo:

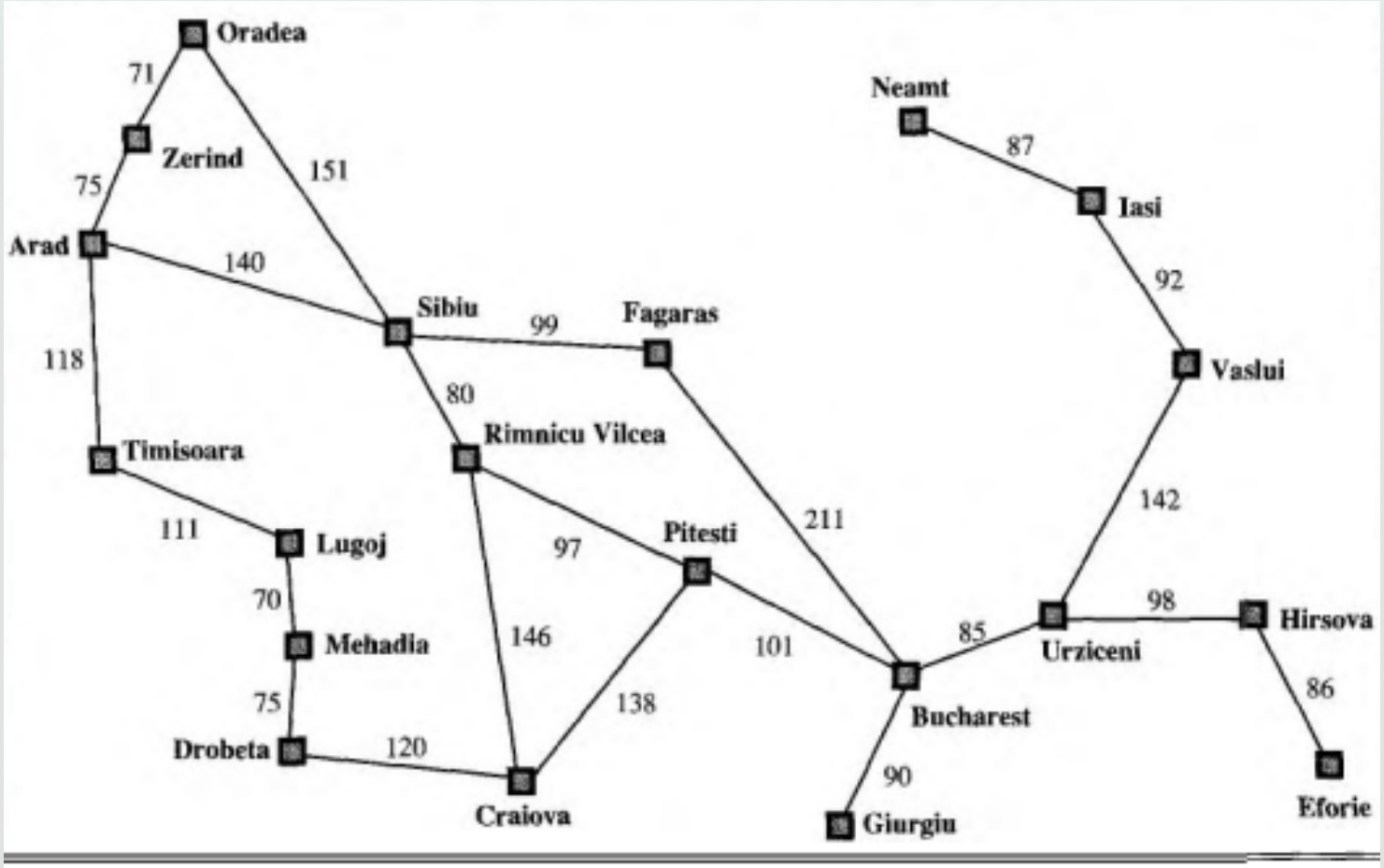


Figure 3.2 A simplified road map of part of Romania.

Profundidade Iterativa

- Variação da Busca em Profundidade Limitada

Profundidade Iterativa

- Variação da Busca em Profundidade Limitada
  - Utilizado se a posição do objetivo é desconhecida.

Profundidade Iterativa

- Variação da Busca em Profundidade Limitada
  - Utilizado se a posição do objetivo é desconhecida.
  - Combina vantagens de BP e BL:
    - Espaço de memória da BP;
    - Encontra o objetivo mais próximo da raiz (mais raso).

Profundidade Iterativa

- Método:

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
    inputs: problem, a problem

    for depth ← 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH(problem, depth)
        if result ≠ cutoff then return result
```

Profundidade Iterativa

- Alguns estados são gerados várias vezes.
  - Desperdício?

Profundidade Iterativa

- Alguns estados são gerados várias vezes.
  - Desperdício?
    - Não é importante.
    - Maioria dos estados nos níveis mais baixos, repetição dos níveis mais rasos.

Profundidade Iterativa

- Alguns estados são gerados várias vezes.
  - Desperdício?
    - Não é importante.
    - Maioria dos estados nos níveis mais baixos, repetição dos níveis mais rasos.
      - As outras buscas visitam estados em níveis mais baixos que o do objetivo.

Profundidade Iterativa

- Comparativo com Largura:

$$N(\mathrm{IDS}) = (d)b + (d-1)b^2 + \ldots + (1)b^d ,$$
$$N(\mathrm{BFS}) = b + b^2 + \ldots + b^d + (b^{d+1} - b) .$$

- Exemplo:
  - Fator de ramificação b = 10, posição objetivo d = 5:

$$N(\mathrm{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$$
$$N(\mathrm{BFS}) = 10 + 100 + 1,000 + 10,000 + 100,000 + 999,990 = 1,111,100$$

Profundidade Iterativa

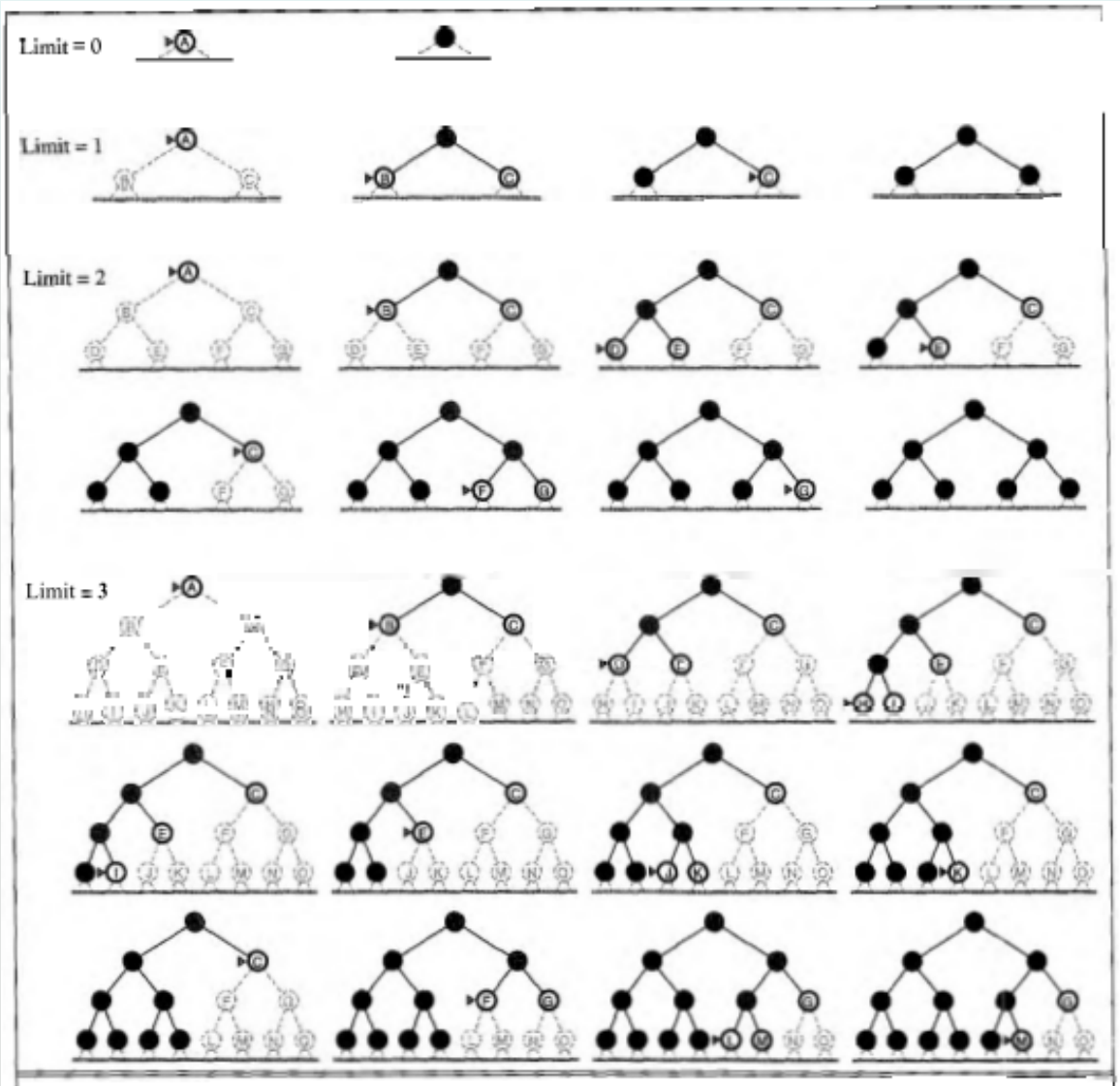


Figure 3.15 Four iterations of iterative deepening search on a binary tree.

Busca Bidirecional

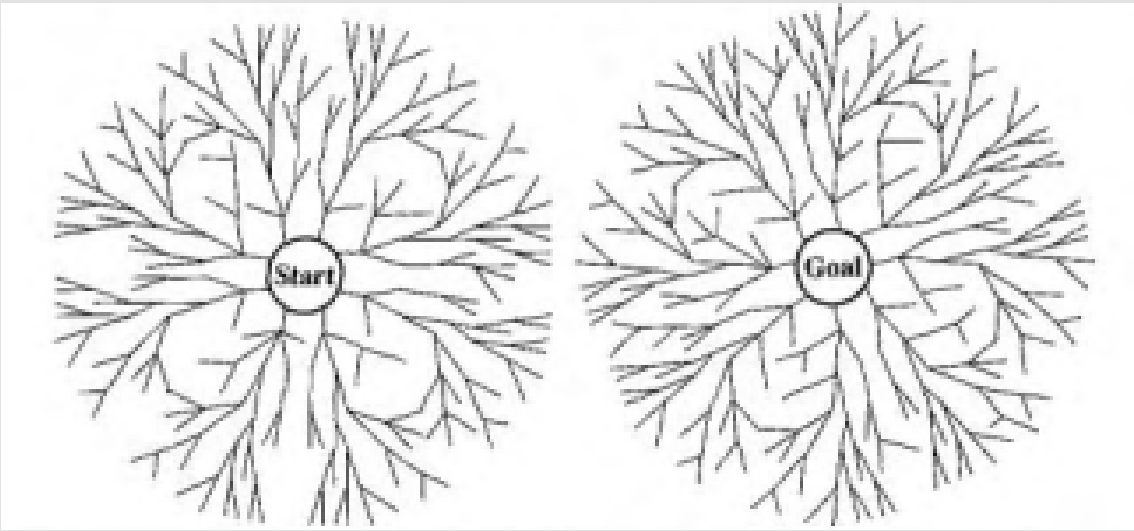
- Busca de caminho mais curto.

Busca Bidirecional

- Busca de caminho mais curto.
  - Motivação:
    - Duas buscas em paralelo.
      - A primeira parte da raiz, a segunda do objetivo.
    - Tempo: $b^{d/2} + b^{d/2} << b^{d}$;

Busca Bidirecional

- Busca de caminho mais curto.
  - Motivação:
    - Duas buscas em paralelo.
      - A primeira parte da raiz, a segunda do objetivo.
    - Tempo: $b^{d/2} + b^{d/2} << b^{d}$;



Busca Bidirecional

- Nem sempre é aplicável;

Busca Bidirecional

- Nem sempre é aplicável;
- Uma das árvores em memória. Espaço: $O(b^{d/2})$
  - Maior deficiência da Busca Bidirecional;

Busca Bidirecional

- Nem sempre é aplicável;
- Uma das árvores em memória. Espaço: $O(b^{d/2})$
  - Maior deficiência da Busca Bidirecional;
- Pode ser ótima:
  - Se as duas buscas forem em largura e
  - os passos tiverem custo fixo.

Comparação das Buscas

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[a]	Yes[a,b]	No	No	Yes[a]	Yes[a,d]
Time	$O(b^{d+1})$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^{d+1})$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]

Figure 3.17 Evaluation of search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: [a] complete if b is finite; [b] complete if step costs $\geq \epsilon$ for positive ϵ; [c] optimal if step costs are all identical; [d] if both directions use breadth-first search.

Buscas Informadas

Melhor Primeiro

- Variação da Subida de Encosta;
  - Diferença sutil:
    - Os nós são ordenado APÓS sua adição à fila.
    - A cada passo, expande-se o MELHOR caminho.

Melhor Primeiro

- Exercício:
  - Comparação Subida de Encosta e Melhor Primeiro.
    - Encontrar o menor caminha de Oradea p/ Craiova.

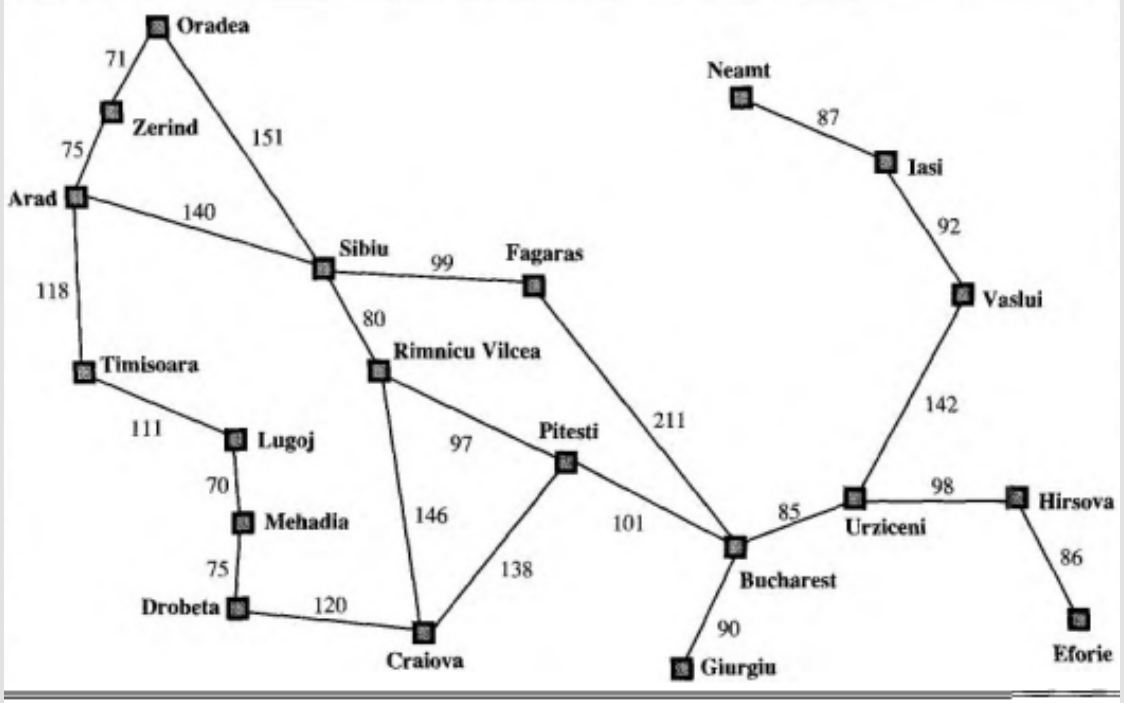
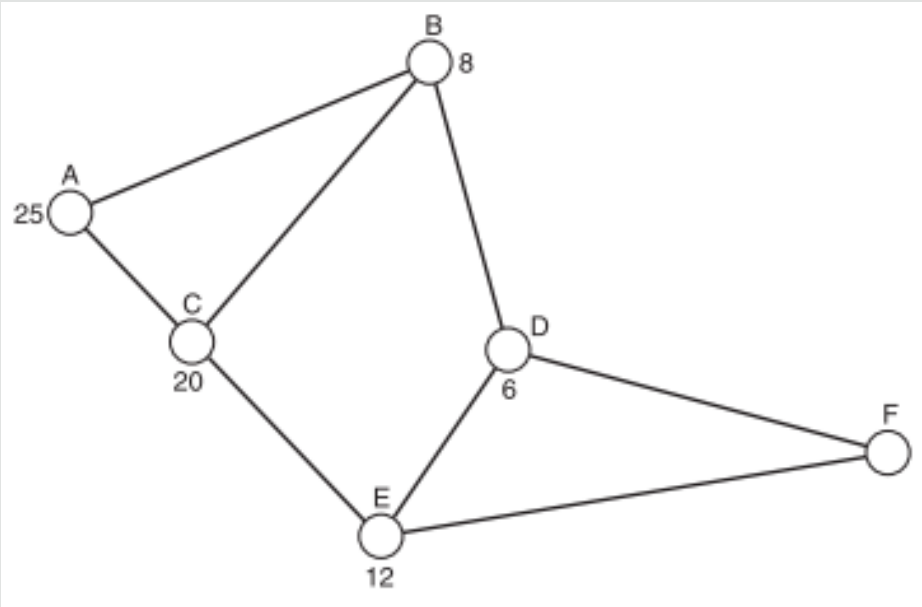
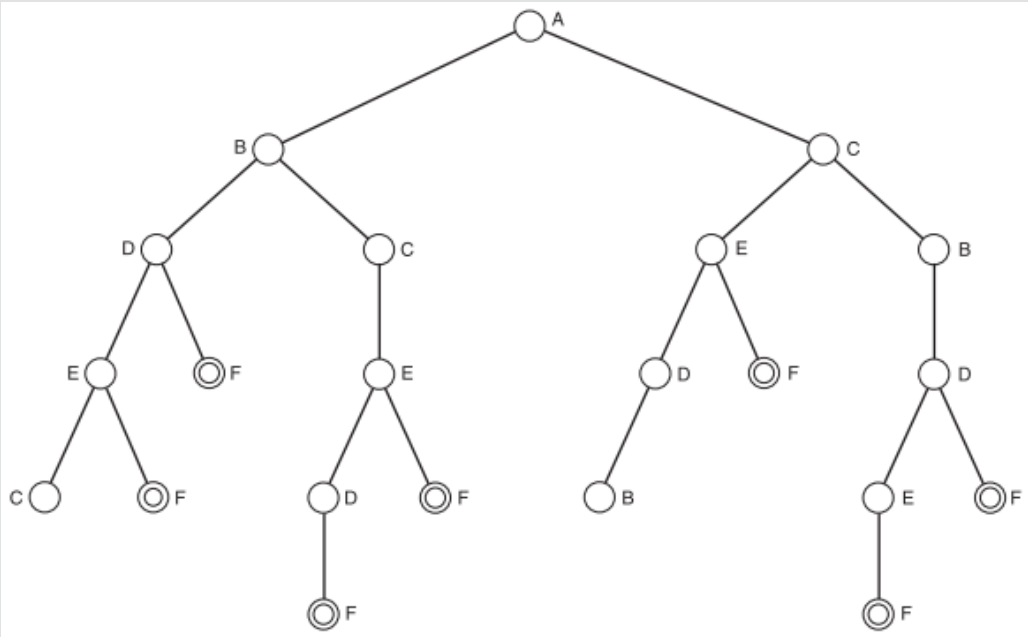


Figure 3.2 A simplified road map of part of Romania.

Melhor Primeiro

- Exercício:
  - Comparação Subida de Encosta e Melhor Primeiro.
    - Encontrar o menor caminho até F.





Melhor Primeiro

- Subida de Encosta vs Melhor Primeiro.

Table 4.4 Analysis of hill climbing

Step	State	Queue	Notes
1	A	(empty)	The queue starts out empty, and the initial state is the root node, which is A.
2	A	B,C	The successors of A are sorted and placed on the queue. B is placed before C on the queue because it is closer to the goal state, F.
3	B	C	
4	B	D,C,C	
5	D	C,C	
6	D	F,E,C,C	F is placed first on the queue because it is closest to the goal. In fact, it is the goal, as will be discovered in the next step.
7	F	E,C,C	SUCCESS: Path is reported as A,B,D,F.

Table 4.5 Analysis of best-first search of tree shown in Figure 4.4

Step	State	Queue	Notes
1	A	(empty)	The queue starts out empty, and the initial state is the root node, which is A.
2	A	B,C	The successors of the current state, B and C, are placed in the queue.
	A	B,C	The queue is sorted, leaving B in front of C because it is closer to the goal state, F.
3	B	C	
4	B	D,C,C	The children of node B are added to the front of the queue.
5	B	D,C,C	The queue is sorted, leaving D at the front because it is closer to the goal node than C.
6	D	C,C	Note that although the queue appears to contain the same node twice, this is just an artifact of the way the search tree was constructed. In fact, those two nodes are distinct and represent different paths on our search tree.
7	D	E,F,C,C	The children of D are added to the front of the queue.
8	D	F,E,C,C	The queue is sorted, moving F to the front.
9	F	E,C,C	SUCCESS: Path is reported as A,B,D,F.

Busca com Limite Superior

- Variação da BL com uso de uma heurística.

Busca com Limite Superior

- Variação da BL com uso de uma heurística.
  - Mantém somente os k melhores a cada passo.

Busca com Limite Superior

- Variação da BL com uso de uma heurística.
  - Mantém somente os k melhores a cada passo.
    - A heurística é usada para classificar os nós.
      - Normalmente, usa-se os caminhos menores ou aqueles que atingem nós com maior valor heurístico (+ proximos do objetivo).

Busca com Limite Superior

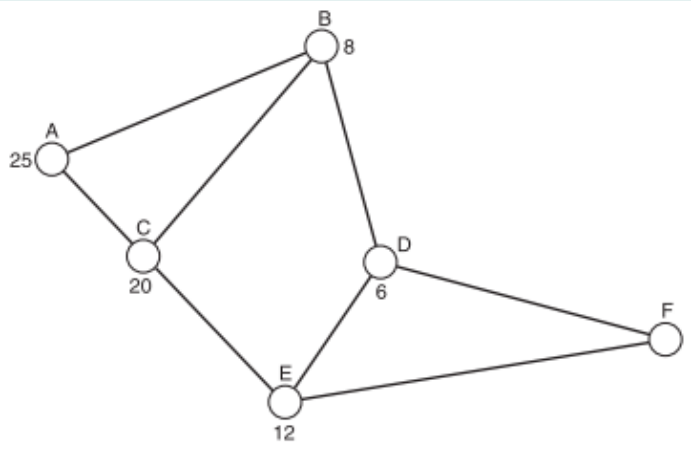
- Variação da BL com uso de uma heurística.
  - Mantém somente os k melhores a cada passo.
    - A heurística é usada para classificar os nós.
      - Normalmente, usa-se os caminhos menores ou aqueles que atingem nós com maior valor heurístico (+ proximos do objetivo).
    - O fator k:
      - Melhora a eficiência e o custo de memória;
      - Pode tornar o método incompleto.

Busca com Limite Superior

- Limite Superior (Beam Search):

```
Function beam ()
{
     queue = [];     // initialize an empty queue
     state = root_node;   // initialize the start state
     while (true)
     {
          if is_goal (state)
               then return SUCCESS
          else
          {
               add_to_back_of_queue (successors (state));
               select_best_paths (queue, n);
          }
          if queue == []
               then report FAILURE;
          state = queue [0]; // state = first item in queue
          remove_first_item_from (queue);
     }
}
```

Busca com Limite Superior



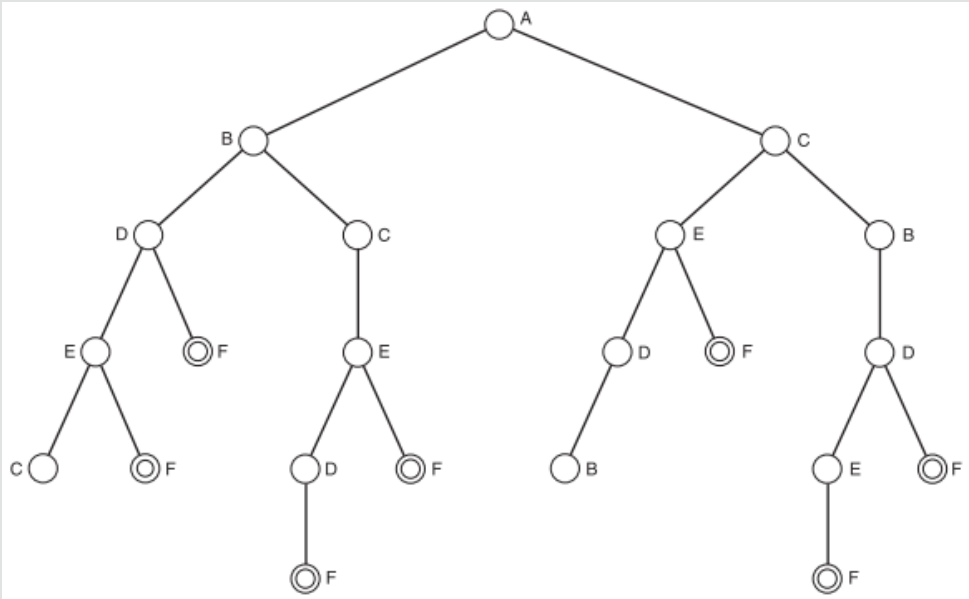


Table 4.6 Analysis of beam search of tree shown in Figure 4.7

Step	State	Queue	Notes
1	A	(empty)	The queue starts out empty, and the initial state is the root node, which is A.
2	A	B,C	The two children of the current node are added to the back of the queue.
3	B	C	
4	B	C,D,C	The two children of B are added to the back of the queue.
5	B	D,C	All but the two best paths are discarded from the queue.
6	D	C	
7	D	C,E,F	The two children of the current node are added to the back of the queue.
8	D	E,F	At this step, C is removed from the queue because we only require the two best paths.
9	E	F	
10	E	F,C,F	The two children of E are added to the back of the queue.
11	E	F,F	The path that leads to C is discarded, in favor of the two better paths, both of which lead to F.
12	F	F	SUCCESS: Path is reported as A,B,D,E,F.

Referências

Russel, S., Norvig, P.; Inteligência Artificial; Editora Campus, Tradução da 2a edição, 2004.

Coppin, B.; Inteligência Artificial; Editora LTC, Tradução da 1a edição, 2010.

Dúvidas